

Software Requirements Specification (SRS)

Project: Vehicle Parking System (VPS)

Version: 1.0

Authors: Himani

Date: 12-09-2026

Status: Draft

Revision history

Version	Date	Author	Change summary	Approval
1.0	12-09-2026	Team 10	Initial draft SRS for Vehicle Parking System	-

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall description
3. External interface requirements
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. System models and diagrams
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for a Vehicle Parking System (VPS). It defines the functional and non-functional requirements, interfaces, and verification criteria for a system that manages parking slots and tracks vehicles across a parking facility.

1.2 Scope

Covers user registration and login, viewing slot availability, booking and releasing slots, vehicle tracking linked to a booking, booking history, and admin functions for managing slots and active bookings. Excludes payment processing, IoT sensor/camera integration for automatic vehicle detection, and physical barrier/gate hardware control.

1.3 Audience

Developers, QA testers, and Course Evaluators.

1.4 Definitions

List of acronyms: VPS (Vehicle Parking System), UI (User Interface), API (Application Programming Interface), DB (Database), ORM (Object-Relational Mapping), FR (Functional Requirement), NFR (Non-Functional Requirement).

2. Overall description

2.1 Product perspective

The VPS is a standalone web application built with Python (Flask) and a relational database (SQLite), used directly by customers and parking lot admin staff through a browser. It is not integrated with external banking, payment, or hardware sensor systems in this version.

2.2 Major product functions (detailed)

- User registration and login
- View available/occupied slots across floors
- Book a parking slot for a vehicle
- Check out / release a slot
- View personal booking history
- Admin: add and remove parking slots
- Admin: view active bookings and force checkout
- Vehicle registration linked to the booking owner

2.3 User roles and characteristics (expanded)

- Customer: registers/logs in, views slot availability, books and releases slots, views own booking history.
- Admin: manages the slot inventory (add/remove) and monitors/forces checkout on active bookings.

2.4 Operating environment

Client: any modern web browser (desktop or mobile). Server: Python 3 with Flask, Flask-SQLAlchemy ORM, SQLite database. Deployable on any OS that supports Python.

2.5 Constraints

Single facility per deployment; no payment gateway integration in this version; session-based authentication only (no OAuth/SSO); floor and slot configuration is managed manually by an admin.

3. External interface requirements

3.1 User interfaces

Web pages built with HTML/CSS and server-rendered templates: registration/login forms, a slot dashboard, a booking form, a history table, and an admin panel.

3.2 Hardware interfaces

None required. The system is accessed from a standard client device (PC or mobile) with a browser; no sensor, camera, or barrier hardware is integrated in this version.

3.3 Software interfaces

Flask-SQLAlchemy ORM interfacing with a SQLite database file; the database layer can be swapped for PostgreSQL/MySQL for production deployment without changing application logic.

3.4 Communications

HTTP between the browser and the Flask server; session cookies maintain login state. HTTPS is recommended for production deployment.

This project includes 17 functional requirements, 5 non-functional requirements, 3 security objectives, and 5 security requirements (see Sections 4, 5, and 5.1).

4. System features (detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow VPS-F-###.

4.1 User Registration & Authentication

Description: Allow a customer to create an account and securely log in.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
VPS-F-001	The system shall allow a new user to register with name, email, and password.	Functional	High	Customer	AC-VPS-F-001: Valid details create an account and redirect to login. Test: TC-Reg-01	Email must be unique
VPS-F-002	The system shall validate that the email is unique during registration and reject duplicates.	Functional	High	Customer	AC-VPS-F-002: Duplicate email shows an error, no duplicate record created. Test: TC-Reg-02	-
VPS-F-003	The system shall hash and securely store user passwords; passwords shall never be stored in plaintext.	Functional	High	Security	AC-VPS-F-003: DB inspection shows only hashed values. Test: TC-Auth-01	Uses Werkzeug password hashing
VPS-F-004	The system shall authenticate a registered user via email and password and start a session.	Functional	High	Customer	AC-VPS-F-004: Correct credentials redirect to dashboard; incorrect credentials show an error. Test: TC-Auth-02	-

4.2 Slot Management (Viewing)

Description: Display slot inventory and status across floors.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
VPS-F-005	The system shall display all parking slots with floor, type,	Functional	High	Customer	AC-VPS-F-005: Dashboard lists all slots	-

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	and current status (available/occupied).				with correct status. Test: TC-Slot-01	
VPS-F-006	The system shall present slots grouped by floor for readability.	Functional	Medium	Customer	AC-VPS-F-006: Slot list is ordered by floor then slot number. Test: TC-Slot-02	Filter-by-floor UI is a future enhancement
VPS-F-007	The system shall prevent a slot marked occupied from being booked again until it is released.	Functional	High	Business	AC-VPS-F-007: A booking attempt on an occupied slot returns an error. Test: TC-Slot-03	-

4.3 Slot Booking

Description: Book an available slot for a vehicle and record the check-in.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
VPS-F-008	The system shall allow a logged-in customer to book an available slot by entering a vehicle number and type.	Functional	High	Customer	AC-VPS-F-008: Booking creates an active record and marks the slot occupied. Test: TC-Book-01	-
VPS-F-009	The system shall register a new vehicle automatically if the vehicle number is not already known, linked to the booking user.	Functional	Medium	Customer	AC-VPS-F-009: A new vehicle number creates a vehicle record. Test: TC-Book-02	-
VPS-F-010	The system shall reuse an existing vehicle record if the vehicle number is already registered.	Functional	Medium	Customer	AC-VPS-F-010: A repeat vehicle number does not duplicate the vehicle record. Test: TC-Book-03	-
VPS-F-011	The system shall timestamp the check-in time at the moment of booking.	Functional	High	Operations	AC-VPS-F-011: The booking record stores a check-in timestamp. Test: TC-Book-04	-

4.4 Vehicle Check-out

Description: Release a booked slot and record the check-out.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
VPS-F-012	The system shall allow a customer to check out their vehicle, releasing the slot and recording the check-out time.	Functional	High	Customer	AC-VPS-F-012: Slot status returns to available and the booking is marked completed. Test: TC-Checkout-01	-
VPS-F-013	The system shall prevent a user from checking out a booking that does not belong	Functional	High	Security	AC-VPS-F-013: An unauthorised checkout	-

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	to them, unless the user is an admin.				attempt is rejected. Test: TC-Checkout-02	

4.5 Booking History

Description: Let a customer review their past and active bookings.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
VPS-F-014	The system shall let a customer view their full booking history, including past and active bookings.	Functional	Medium	Customer	AC-VPS-F-014: History page lists all bookings with check-in/out times and status. Test: TC-Hist-01	-

4.6 Admin Slot Management

Description: Let an admin maintain the slot inventory and monitor active bookings.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
VPS-F-015	The system shall allow an admin to add a new parking slot with slot number, floor, and vehicle type.	Functional	High	Admin	AC-VPS-F-015: A new slot appears in the slot list as available. Test: TC-Admin-01	-
VPS-F-016	The system shall allow an admin to remove a slot only if it is currently available (not occupied).	Functional	Medium	Admin	AC-VPS-F-016: Removal of an occupied slot is blocked with an error message. Test: TC-Admin-02	-
VPS-F-017	The system shall allow an admin to view all active bookings and force a checkout if required.	Functional	High	Admin	AC-VPS-F-017: A forced checkout releases the slot and updates the booking status. Test: TC-Admin-03	-

5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans. IDs VPS-NF-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
VPS-NF-001	The slot dashboard shall load within 2 seconds for up to 200 slots under normal load.	Performance	High	Measured page load time <= 2s in test environment. Test: TC-Perf-01
VPS-NF-002	All core actions (register, book, checkout) shall be reachable within 3 clicks from the dashboard.	Usability	Medium	Navigation review confirms <= 3 clicks. Test: TC-UX-01
VPS-NF-003	The system shall keep slot status consistent with active booking state at all times.	Reliability	High	DB consistency check shows no occupied slot without an active booking. Test: TC-Rel-01

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
VPS-NF-004	Session data shall expire on logout and be inaccessible without re-authentication.	Security	High	Accessing the dashboard after logout redirects to login. Test: TC-Sec-01
VPS-NF-005	The codebase shall separate models, routes, and templates to support future feature additions.	Maintainability	Medium	Code review confirms separation of concerns. Checklist: CR-01

5.1 Security

5.1.1 Security Objectives

- Ensure user credentials (passwords) are never stored or transmitted in plaintext.
- Ensure only authenticated users can access booking and account functionality.
- Ensure only authorised admin users can perform slot management and forced checkouts.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
VPS-SR-001	The system shall hash passwords using a secure one-way hashing algorithm before storing them.	Security	High	AC-VPS-SR-001: DB shows only hashed passwords. Test: TC-Sec-02
VPS-SR-002	The system shall restrict admin routes (slot management, forced checkout) to users with the admin role.	Security	High	AC-VPS-SR-002: A non-admin request to an admin route is redirected. Test: TC-Sec-03
VPS-SR-003	The system shall invalidate session data on logout.	Security	High	AC-VPS-SR-003: Dashboard access after logout requires re-login. Test: TC-Sec-01
VPS-SR-004	The system shall validate and sanitise form inputs before using them in database queries.	Security	High	AC-VPS-SR-004: Malformed/malicious input is rejected or safely escaped. Test: TC-Sec-04
VPS-SR-005	The system shall ensure a booking's checkout action can only be performed by its owner or an admin.	Security	High	AC-VPS-SR-005: Checkout by a non-owner, non-admin user is rejected. Test: TC-Checkout-02

6. Quality attributes & Acceptance tests

Exit criteria for acceptance: all high-priority functional requirements implemented and verified, no critical NFR failures, and the RTM shows all listed test cases passed.

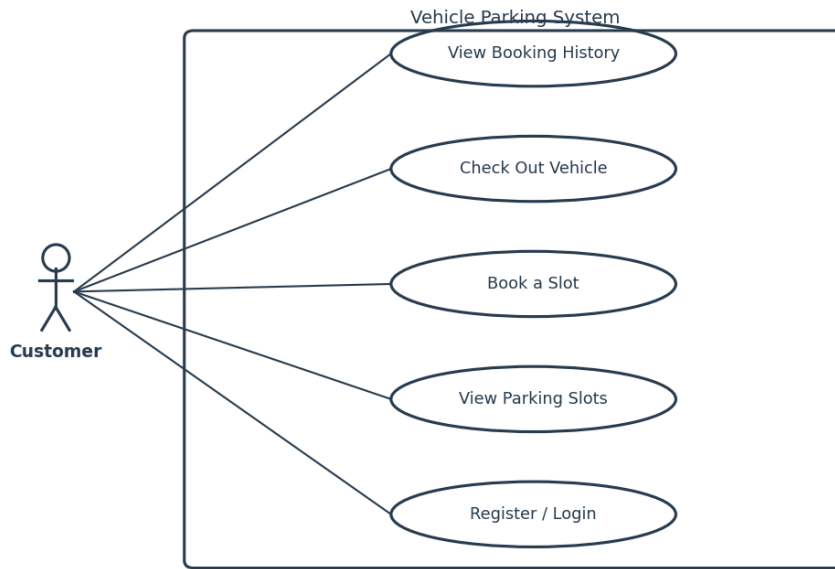
Acceptance test suites: Registration & Authentication, Slot Booking, Check-out, Admin Slot Management, Security, and Usability.

7. System models and diagrams

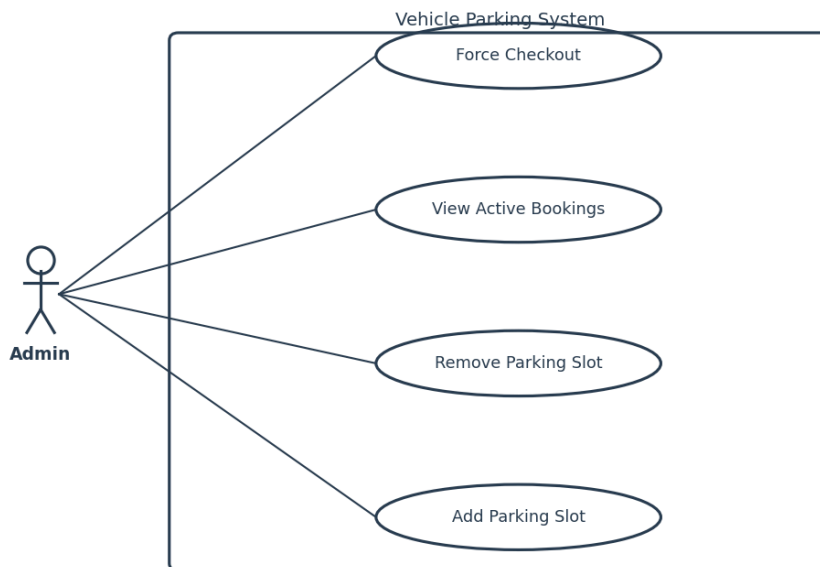
7.1 UML Use-Case diagrams

Diagram 1 shows the Customer actor's use cases; Diagram 2 shows the Admin actor's use cases.

Use Case Diagram 1: Customer



Use Case Diagram 2: Admin



8. Requirements Traceability Matrix (RTM)

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
VPS-F-003	Hash passwords	4.1	AuthModule	TC-Auth-01	N	-

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
VPS-F-004	Authenticate user	4.1	AuthModule	TC-Auth-02	N	-
VPS-F-007	Block double booking	4.2	SlotModule	TC-Slot-03	N	-
VPS-F-008	Book a slot	4.3	BookingModule	TC-Book-01	N	-
VPS-F-012	Check out vehicle	4.4	BookingModule	TC-Checkout-01	N	-
VPS-F-013	Restrict checkout to owner/admin	4.4	BookingModule	TC-Checkout-02	N	-
VPS-F-015	Add slot (admin)	4.6	AdminModule	TC-Admin-01	N	-
VPS-F-017	Force checkout (admin)	4.6	AdminModule	TC-Admin-03	N	-
VPS-NF-001	Dashboard response time	5	WebUI	TC-Perf-01	N	-
VPS-NF-004	Session expiry on logout	5	AuthModule	TC-Sec-01	N	-
VPS-SR-002	Admin route restriction	5.1.2	AdminModule	TC-Sec-03	N	-